Telecom Network Intelligence System

1. Introduction

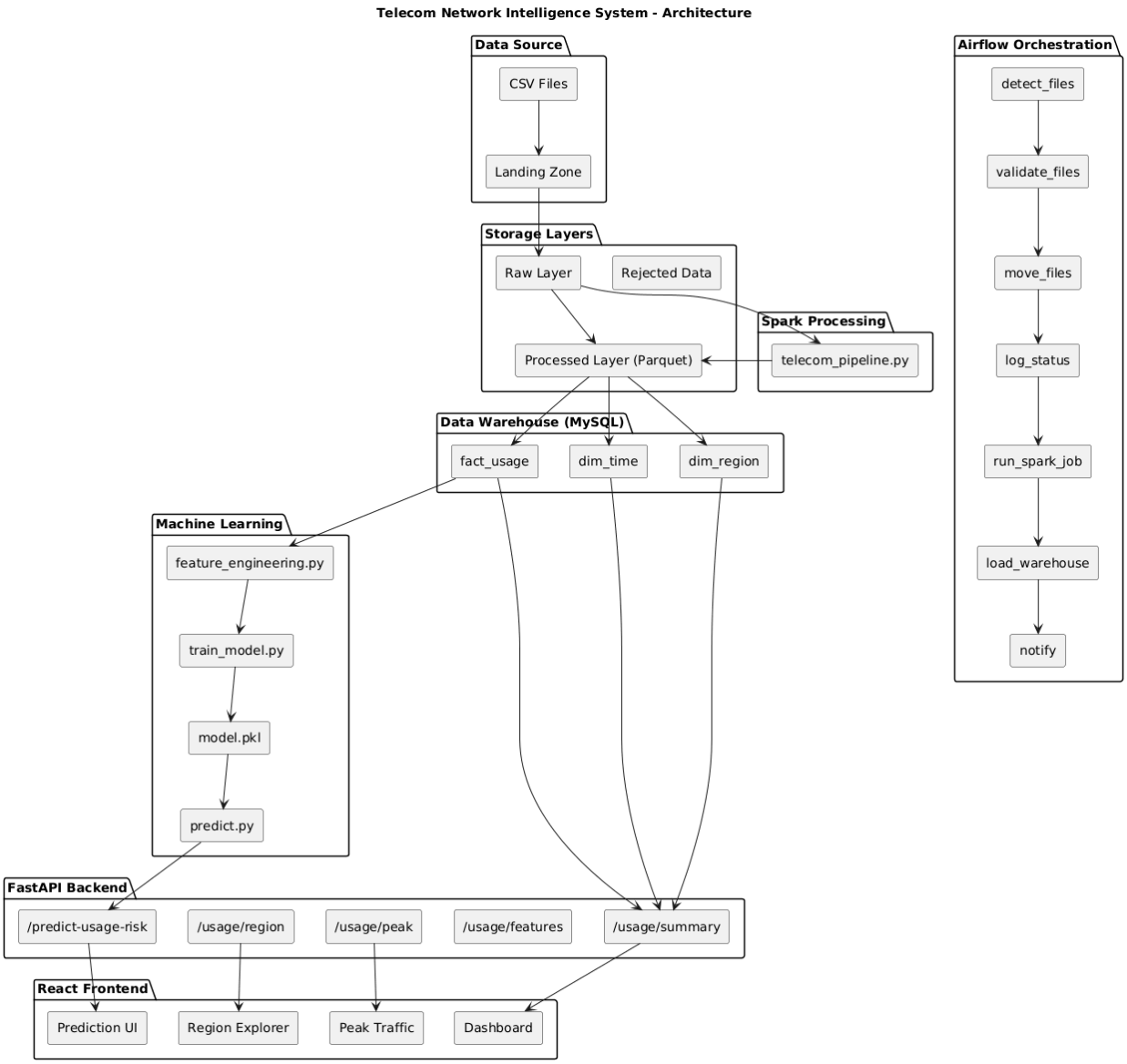
This project implements an end-to-end Telecom Network Intelligence System designed to process, analyze, and predict telecom network usage patterns. The system integrates data engineering, backend API development, frontend visualization, and machine learning to provide actionable insights into network congestion and anomalies.

The system processes telecom activity data (calls, SMS, and internet usage) and enables both analytical reporting and predictive capabilities through a structured pipeline.

2. Overall Architecture

The system follows a layered architecture:

1. Data Ingestion Layer (CSV input)
2. Orchestration Layer (Airflow)
3. Processing Layer (Spark)
4. Storage Layer (Raw → Processed → Warehouse)
5. Backend Layer (FastAPI)
6. Frontend Layer (React)
7. Machine Learning Layer

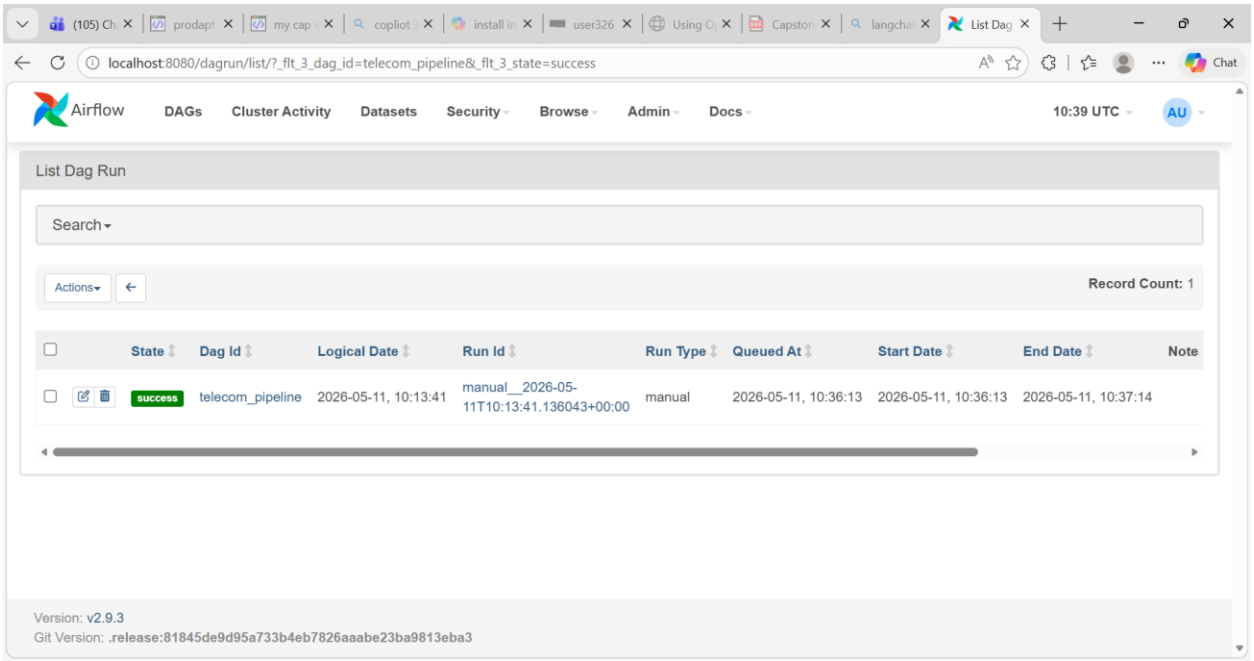


3. Data Flow

The data flows through the system as follows:

1. CSV files are placed in the **landing zone**
2. Airflow detects and validates files
3. Valid files are moved to the **raw layer**
4. Spark processes the data and writes it to the **processed layer (Parquet)**
5. Processed data is loaded into the **MySQL data warehouse**
6. FastAPI exposes analytics APIs

7. React dashboard consumes APIs
8. ML model uses warehouse data to predict congestion risk



4. Data Storage Layers

4.1 Raw Layer

- Format: CSV
- Purpose: Store original input data
- Reason: Easy ingestion and debugging

4.2 Processed Layer

- Format: Parquet
- Purpose: Optimized storage for analytics
- Benefits:
    - Columnar storage
    - Compression
    - Faster querying

4.3 Warehouse Layer

- Database: MySQL
- Schema: Star Schema

5. Warehouse Schema Design

The warehouse follows a **Star Schema** design for efficient analytics.

Fact Table

fact_usage

- usage_id
- time_id
- region_id
- call_count
- sms_count
- internet_mb

Dimension Tables

dim_time

- time_id
- date
- hour
- day
- month
- weekday

dim_region

- region_id
- region_name
- city

```
SELECT r.region_name, t.hour, SUM(f.call_count)
FROM fact_usage f
JOIN dim_time t ON f.time_id = t.time_id
```

```
JOIN dim_region r ON f.region_id = r.region_id
GROUP BY r.region_name, t.hour;
```

6. Batch vs Streaming Decision

Data Type	Category	Reason
Network logs	Streaming	Real-time requirement
Daily usage	Batch	Periodic analysis
Billing reports	Batch	Monthly
Congestion alerts	Streaming	Immediate action
Dashboard analytics	Batch	Aggregated data

The system is designed as a **batch pipeline**, but can be extended to streaming using Kafka.

7. Backend API Design

The backend is implemented using FastAPI.

APIs:

- /usage/summary — overall metrics
- /usage/region/{region} — region drill-down
- /usage/peak — peak analysis
- /predict-usage-risk — ML prediction

All APIs use structured JSON responses and Pydantic models.

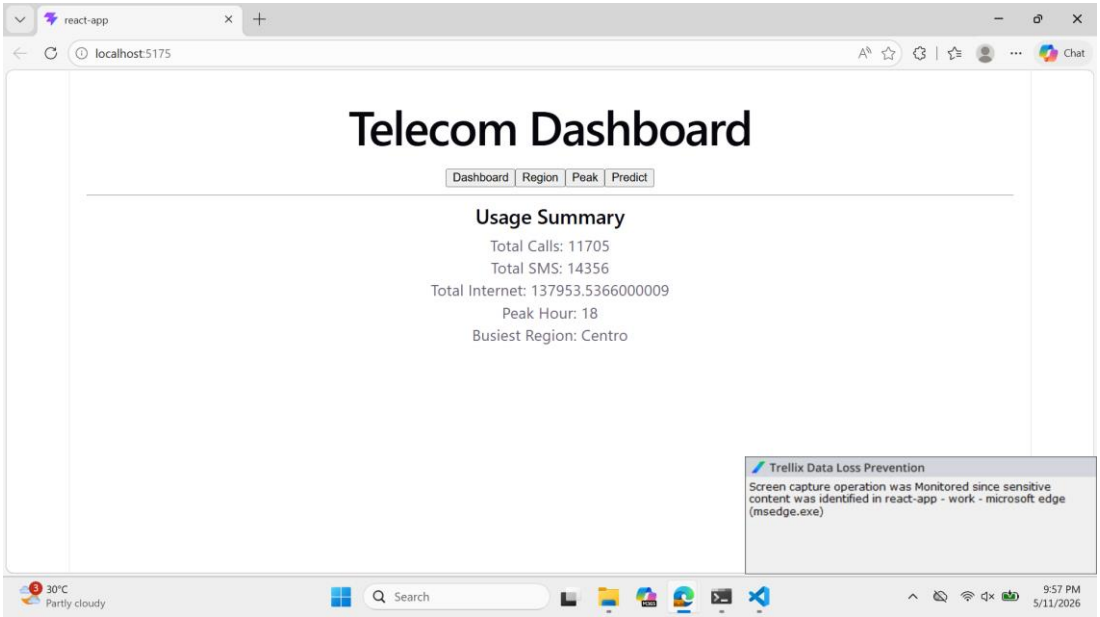
8. Frontend Design

The frontend is built using React with Vite.

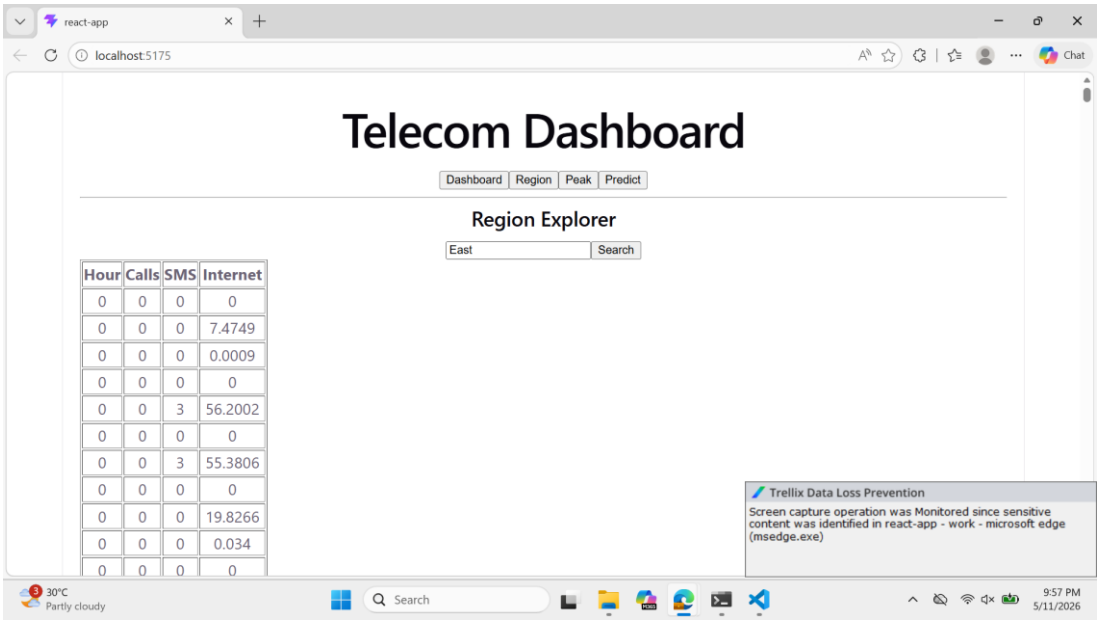
Modules:

1. Usage Dashboard
2. Region Explorer
3. Peak Traffic
4. Prediction Module
5.

Dashboard

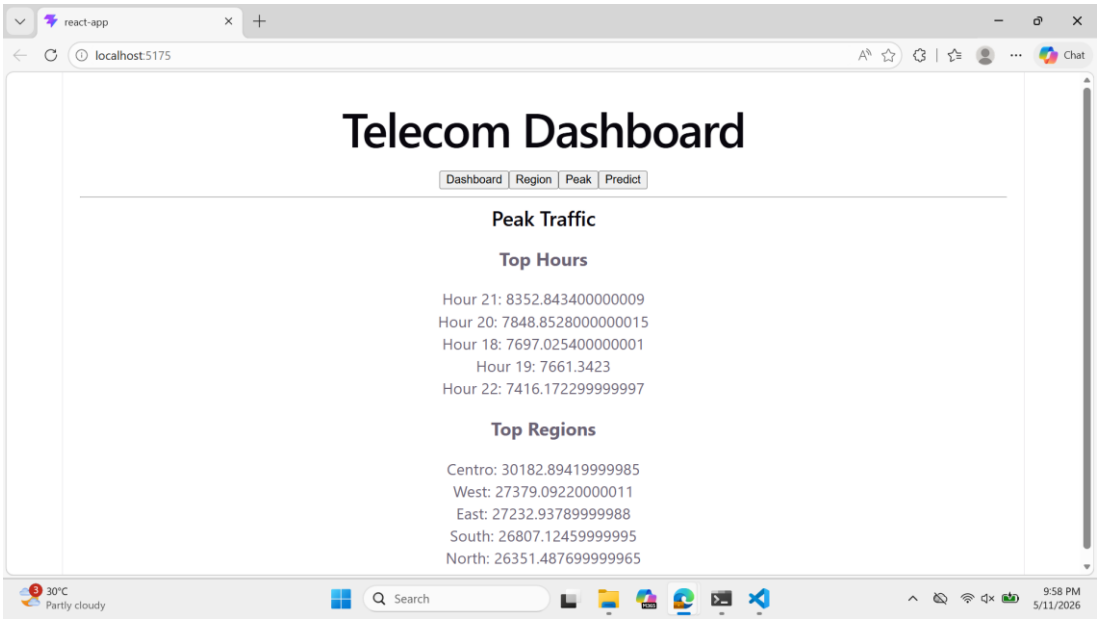
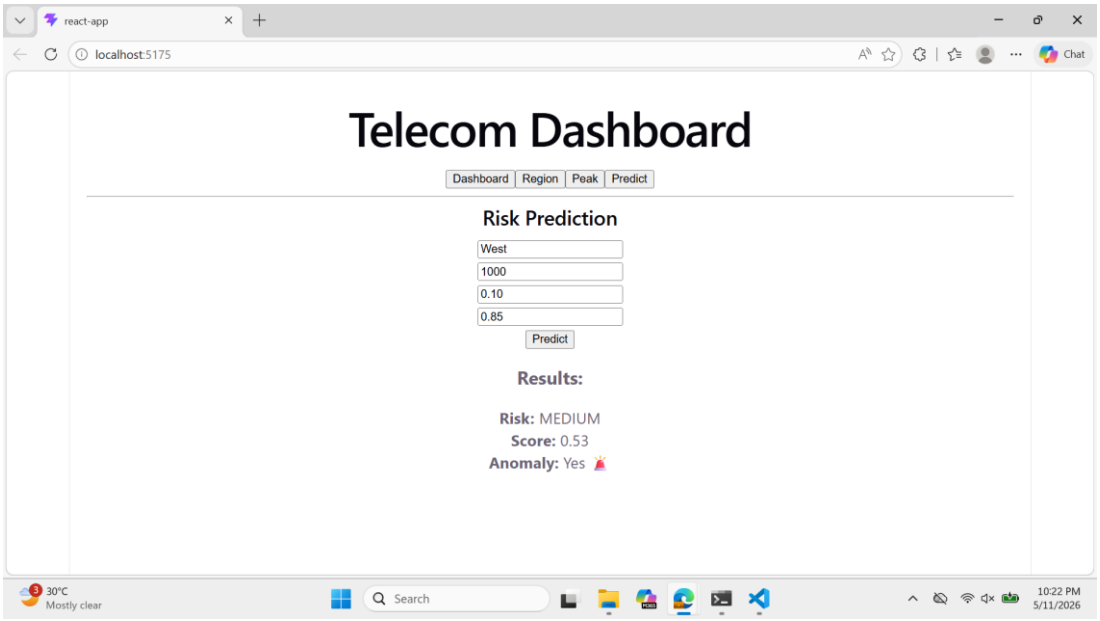


Region



Hour	Calls	SMS	Internet
0	0	0	0
0	0	0	7.4749
0	0	0	0.0009
0	0	0	0
0	0	3	56.2002
0	0	0	0
0	0	3	55.3806
0	0	0	0
0	0	0	19.8266
0	0	0	0.034
0	0	0	0

Peak



Prediction